Validation-First Modeling and Competition-Side XGBoost Exploration for the Kaggle Spaceship Titanic Benchmark

Dingrong Xue, Fan Xie, and Shijun Cheng
Artificial Intelligence Programme, Department of Computer Science
Faculty of Science and Technology
Beijing Normal-Hong Kong Baptist University (BNBU), Zhuhai, China
Team EAP_Hater@MLW; Corresponding author: Dingrong Xue
u430034083@mail.bnbu.edu.cn

***Abstract*—This paper reports our final AI3023 course project on the Kaggle *Spaceship Titanic* competition. The task is a binary classification problem on mixed-type tabular data: predict whether a passenger was transported to an alternate dimension using demographics, cabin assignments, spending behavior, and travel-group identifiers. We deliberately separate two goals that are often conflated in competition projects: obtaining a competitive leaderboard result and producing a defensible machine learning study. To do this, we benchmark five machine learning models under a repeated group-aware validation protocol, build an EDA-driven feature space, and separately analyze a competition-side XGBoost branch that produced the team's highest public score. The clean local benchmark identifies CatBoost as the strongest scientific anchor, while XGBoost offers the best speed-performance compromise and ultimately yields the best public leaderboard result through a more aggressive compact-feature branch. We also show, through shared-audit subgroup disagreement analysis, that the public-best XGBoost branch is specialized rather than uniformly superior to the cleaner CatBoost line. In the public leaderboard snapshot downloaded on May 17, 2026, our team `EAP_Hater@MLW` ranked 97th out of 2240 teams with a score of 0.81716, while our best clean single-model submission ranked 238th with a score of 0.81061. The main conclusion is that for a highly studied benchmark such as *Spaceship Titanic*, rigorous validation, ablation, and reproducibility are more important to the final analysis than optimizing solely for marginal public-leaderboard gains.**

***Index Terms*—Kaggle, tabular classification, CatBoost, XGBoost, feature engineering, cross-validation, reproducibility**

I. Introduction

Working on the AI3023 course project means entering a Kaggle competition at the same time as handing in a full machine learning report, bundled code, and slides. One side pushes speed: checking an open leaderboard and reacting quickly. The other side values care: clear methods, repeatable results, fair comparisons, and reasoning behind design choices. When both shape decisions, the outcome is not defined by score alone. With a dataset such as *Spaceship Titanic*, public rankings matter, but they do not completely define success.

This difference carried real weight in how we approached the project. Public notebooks, small feature tweaks, and leaderboard-oriented methods already fill the competition space. Because of this, a larger public score is not proof of stronger science. We therefore treated the project first as a test of ideas through models, while still recording the path that led to the team's best public outcome.

The paper makes four contributions:

1) A fresh feature set is built through EDA, using only the contest files. Original columns are combined with cabin, passenger-group, and spending features, with no outside information added.
2) Five models are tested under a single validation setup, so differences in accuracy and computational demand can be compared fairly.
3) Repeated group-aware testing is used before promotion decisions. Early exploration stays separate from later go-ahead checks.
4) The team's highest-scoring XGBoost setup is rebuilt step by step, tying the final public result to our own testing path rather than leaving it as a lone leaderboard trophy.

This framing fits *Spaceship Titanic* particularly well. The dataset is limited in size, semantically structured, and widely examined, so nearby validation splits do not always match what appears online in the rankings. We therefore treat the leaderboard not as truth itself, but as one piece of evidence among others.

II. Related Work

Tree ensembles are often the strongest tools for tabular classification, particularly boosted variants such as XGBoost, LightGBM, and CatBoost [2]–[4]. Their strength lies in finding nonlinear patterns across variables and handling uneven feature distributions without excessive preprocessing. Random Forest provides a useful averaging baseline [5], while Logistic Regression remains important as a straight-line reference model [6].

Optuna appears frequently when practitioners search for hyperparameters in competition settings [7]. Still, using a tuning tool does not automatically make a workflow solid in academic terms. The real weight lies in whether the tuning goal is steady, how validation checks are run, and how decisions to move

TABLE I
DATASET SUMMARY AFTER INITIAL INSPECTION

Statistic	Value
Training rows	8,693
Test rows	4,277
Raw predictive columns	13
Positive class rate	50.36%
Largest missing rate	2.50%
Median age	27
Zero-spend rate	42.02%
Multi-passenger rate	44.73%
Group-homogeneous rate	87.18%

a model forward are recorded. More broadly, leaderboard methodology has been studied as a reliability issue: Blum and Hardt show that leaning too heavily on public feedback while tweaking models can distort evaluation [8]. That concern lines up closely with our class environment.

Open Kaggle notebooks gave us practical inspiration during the challenge. Representative examples include Cortinhas' full walkthrough and Arunklenin's work on data exploration and feature crafting [9], [10]. Several themes kept appearing: breaking down cabin information helps, grouping passengers makes a difference, and total spending often holds up better than isolated ancillary-spend variables. These resources shaped early thinking, but we did not use them as a substitute for our own testing.

III. Dataset and Exploratory Analysis

A. Dataset Characteristics

The official competition dataset contains 8,693 labeled training entries and 4,277 unlabeled test entries [1]. Its original structure includes 13 predictive columns covering demographics, destination, cryosleep status, cabin code, five spending categories, and passenger names. The outcome column, `Transported`, is almost evenly split, with a positive rate of 50.36% among labeled samples. Missing values appear only lightly: the largest missing rate for any raw feature is 2.50%.

Not every piece of information acts independently. Each `PassengerId` follows a pattern like `gggg_pp`, where the first part identifies passengers traveling as a unit. In our summary, 44.73% of rows belong to multi-passenger clusters. Because shared trips appear so often, skipping that structure during validation would make little sense.

Table I summarizes the core statistics used throughout the paper.

B. Exploratory Data Analysis

Our EDA was not included just to make the report look complete; it shaped how models were chosen. Figure 1 shows two clear facts: some data are missing, though not much, and the target classes are fairly even. Because of that balance, accuracy is a reasonable main evaluation measure, while F1 and ROC-AUC still matter when looking more closely at specific cases.

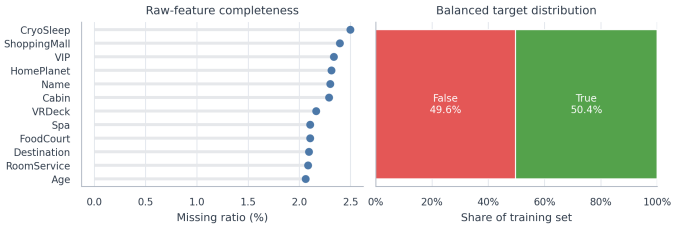

Fig. 1. Data completeness and target balance. The dataset is not sparse, but careful imputation remains necessary.

Figure 2 reveals more curious trends. Age matters in a curved rather than straight-line way, especially when separating children from adults. Spending stands out more sharply: many transported passengers stay close to zero extra cost, which fits the expected behavior of passengers in cryosleep. Home planets and deck levels also differ widely, so category-aware models and deck-position variables are natural choices.

Figure 3 adds another detail. Solo travelers form the largest segment, yet families and small groups appear often enough to matter. Features built around passenger clusters therefore become useful. Spending measures also move together without repeating one another exactly. Total amount paid and individual costs each bring a different angle, so keeping both aggregate and component-level spending preserves information.

IV. Methodology

A. Preprocessing and Feature Engineering

The preprocessing rule was simple: each crafted feature had to come directly from the competition data, with nothing extra and nothing guessed. It also had to hold up when explained aloud. In the end, the feature space was organized into five groups:

1) Original travel and demographic variables: `HomePlanet`, `Destination`, `Age`, `VIP`, and `CryoSleep`.
2) Cabin features: `Deck`, `CabinNum`, `Side`, and deck-side combinations extracted from the cabin code.
3) Passenger-cluster features: `PassengerGroup`, `PassengerNo`, `GroupSize`, and `IsAlone`.
4) Spending features: `TotalSpend`, zero-spend indicators, spend ratios, and log-transformed spending.
5) Rule-consistency features, especially cryosleep-spend consistency indicators.

Table II summarizes these families and the intuition behind them.

B. Validation Protocol

Validation design mattered most. Since passengers in the same group tend to have related outcomes, splitting randomly by row can accidentally reveal connections across folds. We therefore used repeated `StratifiedGroupKFold`, grouping by the first part of `PassengerId`.

Seeds took on different jobs:

- Search seeds {`7, 21, 42, 87, 123`} were used for trying new features and exploring model behavior.

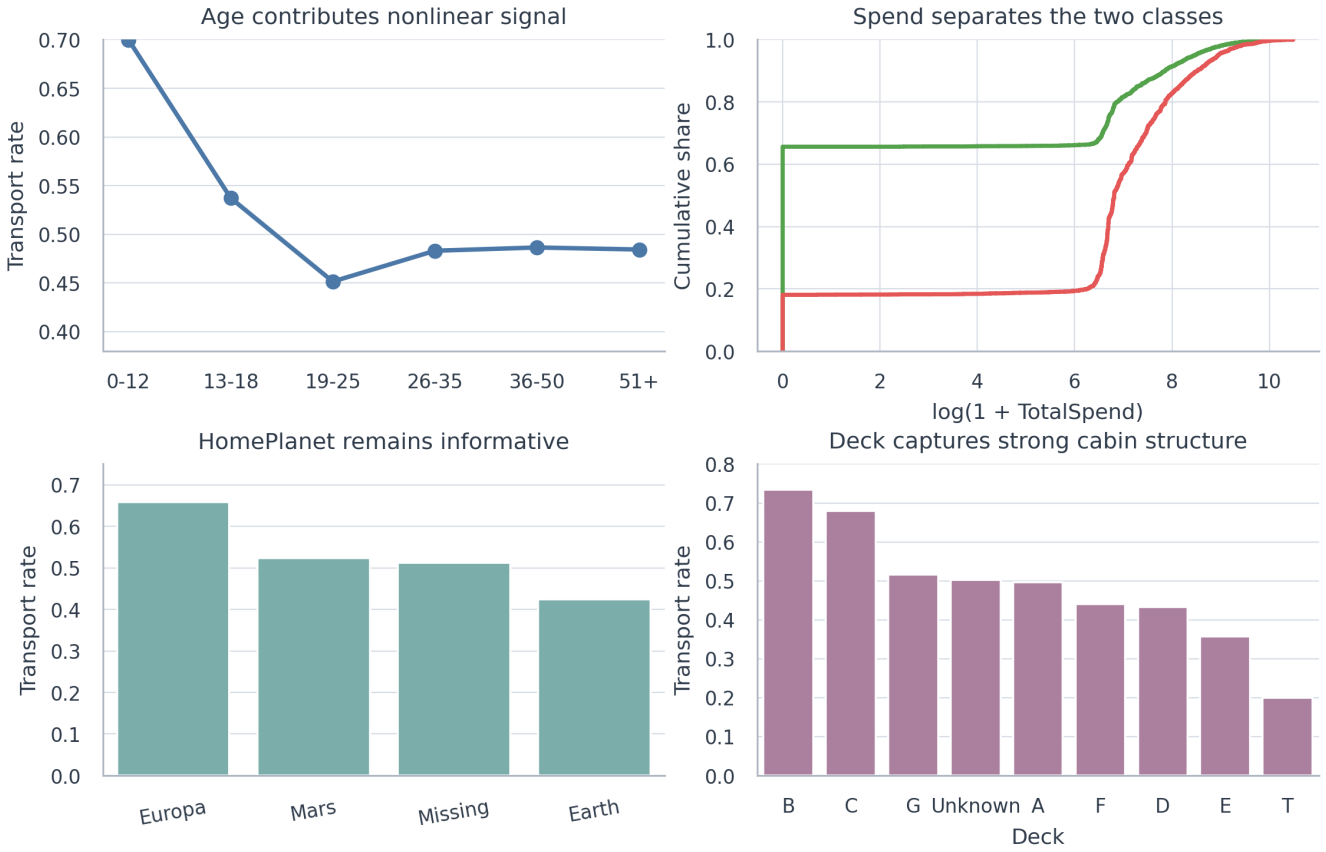

Fig. 2. Core EDA findings. Spending, home planet, and deck have clear predictive value, while age contributes weaker but still meaningful nonlinear structure.

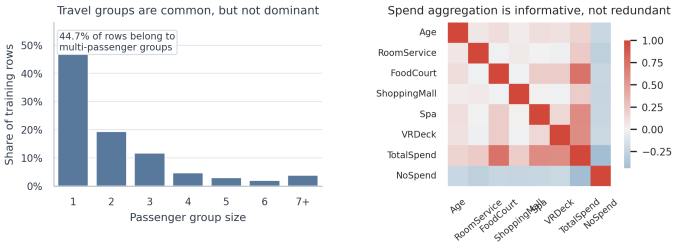

Fig. 3. Passenger-group structure and numeric correlation. These patterns motivate group-aware validation and spend aggregation instead of raw-column-only modeling.

TABLE II
ENGINEERED FEATURE FAMILIES

Family	Rationale and examples
Original demographics	`HomePlanet`, `Destination`, `Age`, `VIP`, `CryoSleep`. These preserve the official schema.
Cabin-derived	`Deck`, `CabinNum`, `Side`, and `DeckSide`. These encode spatial structure inside the ship.
Passenger-group-derived	`PassengerGroup`, `PassengerNo`, `GroupSize`, and `IsAlone`. These model travel-group behavior explicitly.
Spend-derived	`TotalSpend`, `NoSpend`, spend ratios, and `LogTotalSpend`. These summarize luxury-service behavior more cleanly.
Rule-consistency	Cryosleep-spend consistency flags. These encode plausible behavioral constraints.

- Audit seeds `{3, 11, 17, 29, 57, 68, 99, 131}` were reserved for reviewing whether a change should move forward.

A single small gain was not enough. A candidate had to improve out-of-fold accuracy by at least 0.001 on both seed groups, while the audit threshold stayed fixed at 0.5. This strict filter blocked several ideas that looked good at first glance but weakened under closer checking.

C. Model Families and Tuning Strategy

Five models were tested, although the course required only four. Logistic Regression and Random Forest acted as reference points, one linear and one based on grouped decision trees. LightGBM, XGBoost, and CatBoost represented the stronger boosting family commonly seen in structured-data competitions.

The tuning strategy depended on what each model was meant to do. For fair comparison, the validation-first setup kept the feature space fixed and reused the same group-aware split logic. LightGBM and CatBoost were adjusted locally around already solid configurations. XGBoost appeared twice, with different roles:

1) as part of the clean unified benchmark;
2) as a separate competition-side branch focused on climb-

TABLE III
COMPETITION-SIDE XGBOOST BRANCH CONFIGURATION

Component	Setting
Categorical harmonization	Train and test concatenated before categorical imputation and one-hot alignment
Imputation logic	Mean imputation for numeric columns, most-frequent imputation for compact categorical set, plus room-guided filling for `VIP`, `Cabin`, `HomePlanet`, and `Destination`
Compact branch features	27 encoded features before pruning; 15 after pruning
Outlier diagnostic	`IsolationForest` with contamination = 0.003 was evaluated during branch reconstruction but not used in the final submitted training matrix
Class balancing	`KMeansSMOTE`; training rows increased to 8,762
Final XGBoost setting	seed 36086; `max_depth`=5, `n_estimators`=950, `learning_rate`=0.0475, `subsample`=0.96, `colsample_bytree`=0.93, `reg_alpha`=4.582, `reg_lambda`=3.06

ing the public leaderboard.

The competition-side XGBoost branch relaxed some of the strict data rules used in the standard benchmark. Train and test data were combined during categorical alignment. Missing values were filled with help from room-level patterns. Encoded features were trimmed by hand from 27 to 15. `KMeansSMOTE` was used to even out the training set before fitting a regularized XGBoost model. The best public setup used seed 36086, `max_depth`=5, `n_estimators`=950, `learning_rate`=0.0475, `subsample`=0.96, `colsample_bytree`=0.93, `reg_alpha`=4.582, and `reg_lambda`=3.06. This path stands apart because winning mattered more than purity in this branch; not every choice favors clarity, but the branch did improve the public result.

D. Experimental Controls and Reproducibility

Holding the report together meant locking down key choices in the validation-first setup. The same feature families were used each time. The same group-aware splitting logic shaped every test. Thresholds stayed fixed on the audit seeds, and outside prediction files were left out completely. Because everything lined up this way, the benchmark table shows what the models bring, not process noise from unrelated pipelines.

We also kept separate records of tuning steps for the clean benchmark setups. The LightGBM random-search winner pushed audit accuracy up by +0.0031 over the standard starting point. The best CatBoost run edged ahead by +0.0023. These gains appear later as proof that tuning was actually performed rather than only claimed.

The competition-side XGBoost branch was kept apart on purpose. Its goal was to push the public leaderboard score as high as possible under the contest rules. Splitting the tracks helped avoid a common flaw in class projects, where a model built for rankings quietly replaces the clearer scientific baseline.

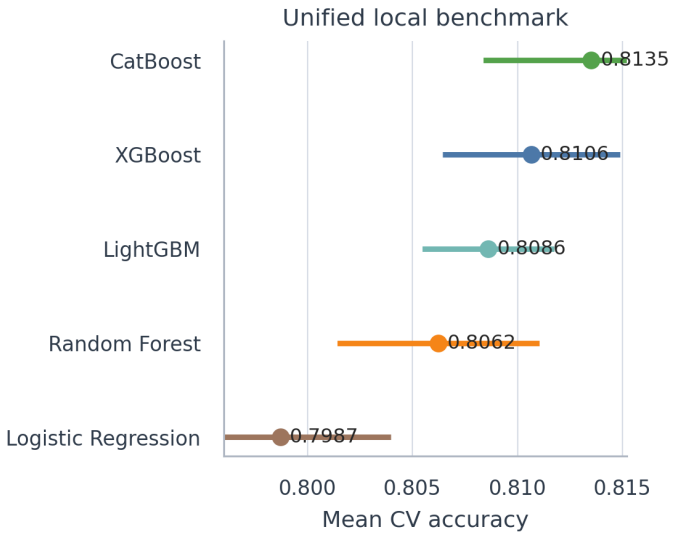

Fig. 4. Mean CV accuracy under the unified benchmark. Boosting clearly dominates the task.

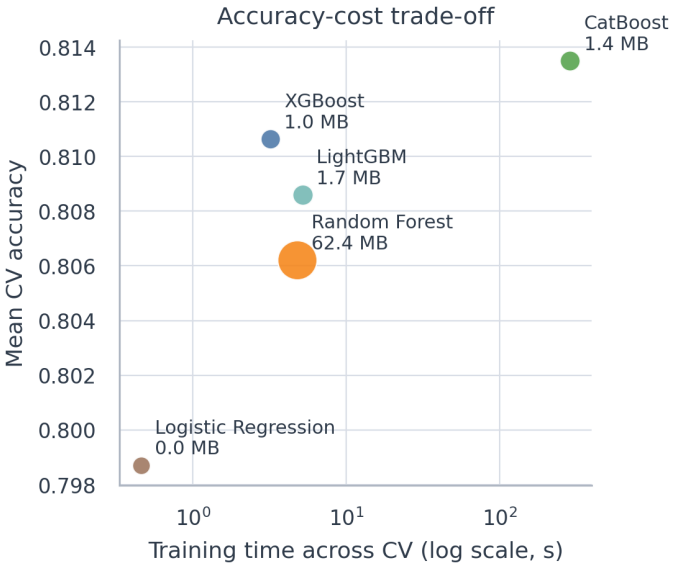

Fig. 5. Accuracy-cost trade-off. CatBoost is strongest locally, while XGBoost is the most attractive speed-performance compromise.

V. EXPERIMENTAL STUDY AND RESULTS

A. Unified Local Benchmark

Table IV reports mean CV accuracy, OOF F1, OOF AUC, training time, inference cost, and artifact size under one benchmark scaffold. The point is not only to rank scores, but also to see which models are practical to train, store, and use.

Figure 4 shows how the models line up at a glance. CatBoost takes the top local position. XGBoost stays close behind while learning much faster. The basic linear model does not capture enough of what matters.

B. Why the Models Behave Differently

The benchmark is useful because it shows more than rankings. It also shows how each model fits the data. Logistic

TABLE IV
UNIFIED LOCAL BENCHMARK UNDER THE SAME FEATURE SET AND VALIDATION REGIME

Model	Mean CV Acc.	OOF F1	OOF AUC	CV Fit Time (s)	Test Infer. (ms/sample)	Artifact Size (MB)
CatBoost	0.8135	0.8143	0.9069	288.32	0.0230	1.37
XGBoost	0.8106	0.8103	0.9034	3.19	0.0061	1.04
LightGBM	0.8086	0.8099	0.9012	5.20	0.0089	1.65
Random Forest	0.8062	0.8013	0.8930	4.79	0.0350	62.40
Logistic Regression	0.7987	0.8047	0.8887	0.46	0.0040	0.01

Regression falls short because the problem contains complex links: cryosleep shapes spending patterns, group travel mixes with cabin details, and cabin codes reveal meaning only after being broken down. Random Forest captures some of these overlaps through branching splits, but it demands more storage while still trailing the boosting methods.

LightGBM and XGBoost fit the task naturally because they detect complex feature links quickly on transformed tabular data. CatBoost performs best locally because the mix of numeric and categorical fields plays directly to its strengths. This matches the earlier EDA: home world, deck level, side, passenger cluster, and spending all shape one another more deeply than numbers alone can show, while still remaining within the range where tree models work well.

Table V summarizes these qualitative fit arguments.

C. Interpretability of the Clean CatBoost Anchor

The uncluttered CatBoost version is also the easiest to follow. Figure 6 ranks the main features from the validation-first CatBoost run. The leading positions go to `TotalSpend`, `Deck`, `HomePlanet`, and `PassengerGroup`. These match closely with what the earlier analysis suggested, even though the model sharpens the ordering. This is why the tidy CatBoost setup works well at the center of the report: its outputs make sense in context rather than relying on hidden adjustments after the fact.

D. Feature-Family Ablation

Each feature group was tested step by step to see which ones actually carried weight. The evaluation used the clean CatBoost line and followed the same group-aware split rules. This made the gaps between performance jumps easier to interpret, rather than assuming that every added feature helped. Figure 7 summarizes the result.

The pattern is not just a count of features accumulating one after another. Basic official data alone reached 0.7973. When cabin details were added, group-aware CV jumped to 0.8125, the largest increase in the study. Group traits barely moved the number afterward. Spend summaries and rule-consistency features did not help in this fixed CatBoost setting and even nudged the average down slightly. This does not make group or spending clues worthless; rather, in this setup, later groups overlap with the strong cabin-based foundation more than they add new signal. The result supports cautious selection rather than random inclusion.

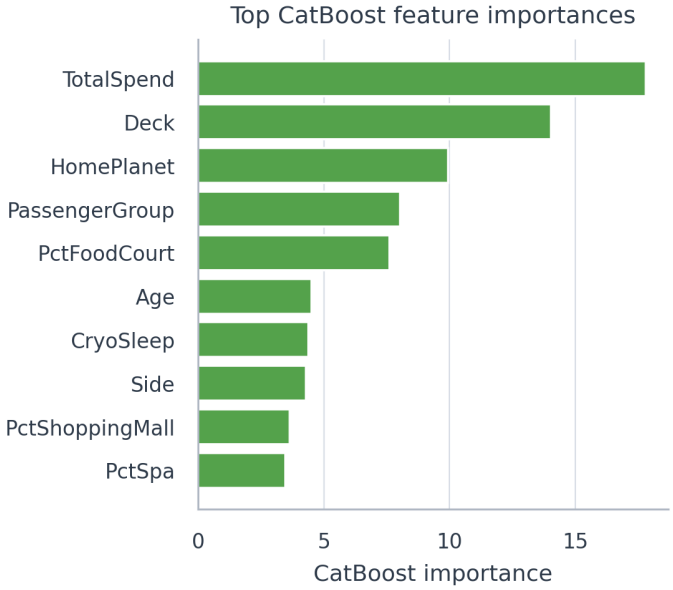

Fig. 6. Top feature importances from the clean CatBoost anchor. The strongest signals align well with the EDA findings.

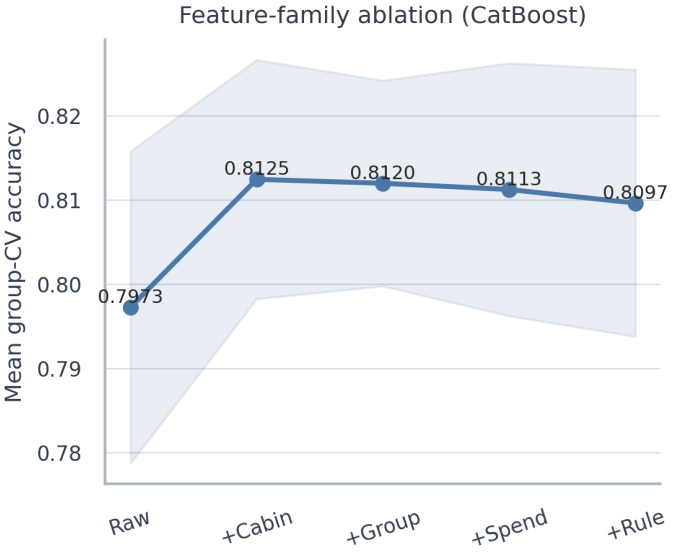

Fig. 7. Cumulative feature-family ablation on the clean CatBoost line. Cabin decomposition produced the largest single improvement.

E. Tuning Evidence on Clean Models

Figure 8 summarizes the audited gains from the two clean-model tuning tracks retained for the final analysis.

TABLE V
QUALITATIVE SUITABILITY OF EACH BENCHMARKED MODEL FOR THIS DATASET

Model	Main strengths on this task	Main weaknesses on this task	Overall suitability
Logistic Regression	Very fast, interpretable, and useful as a sanity-check baseline.	Cannot capture nonlinear cryosleep-spend-group interactions compactly.	Low. Good baseline, weak final model.
Random Forest	Flexible nonlinear baseline with few distributional assumptions.	Large artifact size and weaker score than boosting under the same feature space.	Moderate. Reasonable baseline, not the best trade-off here.
LightGBM	Fast training and strong performance on sparse encoded features.	Small local gains proved unstable under wider audit checks.	High. Strong clean benchmark model.
XGBoost	Best speed-performance compromise and highly competitive competition model.	Sensitive to preprocessing and feature-pruning choices.	High. Strong practical model, especially for the competition-side branch.
CatBoost	Best local accuracy and very natural fit for mixed-type tabular data.	Slower to train; not the top public-score branch.	Very high. Best scientific anchor for the final report.

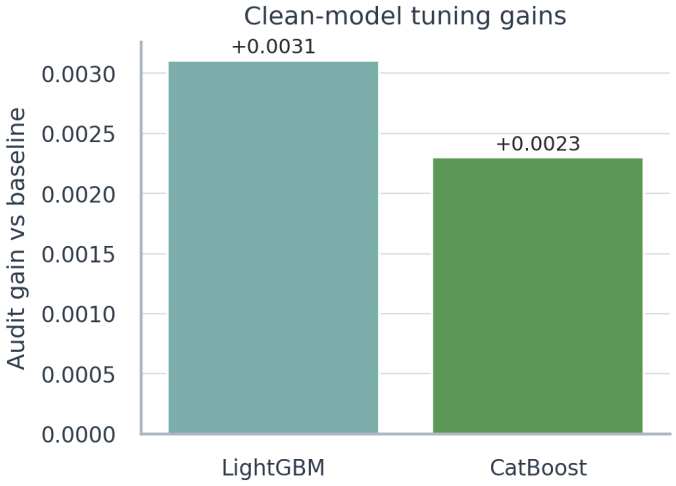

Fig. 8. Audited tuning gains for the clean LightGBM and CatBoost search tracks retained for the final analysis.

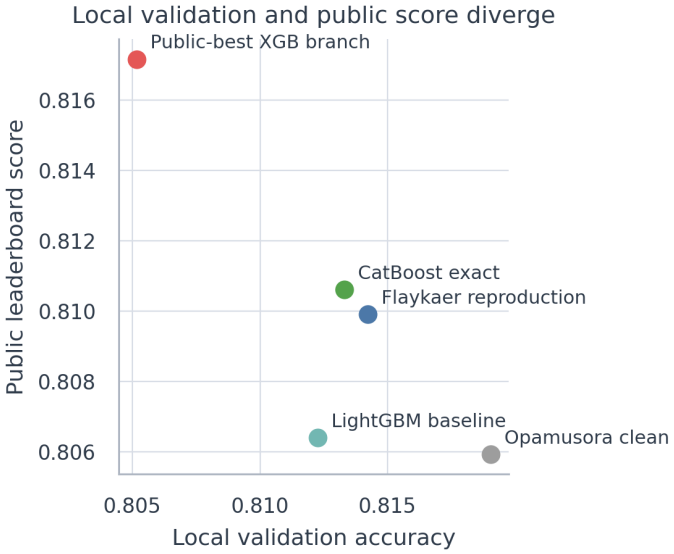

Fig. 9. Local validation does not monotonically predict public leaderboard score. This is why the report emphasizes validation design and ablation rather than public-score optimization alone.

LightGBM produced a small audit gain of +0.0031 through random search with separate audit seeds. CatBoost improved by +0.0023 compared with the clean setup through a narrow set of local tweaks. These shifts are small, but they meet the class requirement for tuning work and fit within normal reporting practice.

Table VI makes the tuning path more concrete. The best LightGBM choice used smaller leaves, deeper explicit tree levels, stronger regularization, and larger leaf limits. The best CatBoost choice also moved toward deeper splits, stronger L_2 settings, modest sampling, and fewer random jumps across features. What stands out is restraint: the search favored balance and control rather than flashy tuning moves.

F. Validation Alignment versus Public Leaderboard

One of the clearest findings is that local scores and public rankings often move apart. Figure 9 makes this point directly. A strong CatBoost candidate reached 0.81905 under internal group-aware validation but dropped to 0.80593 publicly. On another path, the XGBoost branch reached 0.81716 on the public leaderboard while scoring only 0.80516 under our stricter audit. The numbers shift, but not always together.

That mismatch shifted how we worked. We did not accept minor local changes until they passed the audit seeds. Rather than force one story to explain both outcomes, we split the top clean solo model from the leading public entry. The CatBoost setup is simple to understand and scored 0.81061 publicly. A separate XGBoost variation rose higher because it was built for a different goal.

Table VII summarizes the candidate lines that mattered most to this conclusion. The table is useful because it separates three different concepts that are easy to blur together in a Kaggle report: the strongest clean scientific anchor, the strongest local-CV counterexample, and the strongest public-leaderboard branch.

G. The XGBoost Exploration Branch

The highest public score in the team history was not the result of one lone entry. It unfolded along a path built around XGBoost. Figure 10 maps this development. On April 1, 2026, an early XGBoost-only attempt reached 0.79378. By April 10, a revised solo model reached 0.80383. On April 16, an XGBoost-assisted stack lifted the score to 0.81084. Finally, on April 26, the most refined XGBoost branch reached 0.81716.

TABLE VI
KEY HYPERPARAMETER CHANGES RETAINED FROM THE CLEAN TUNING RECORDS

Model	Baseline setting	Tuned winner setting	Audit gain
LightGBM	500 trees, 31 leaves, unlimited depth, `subsample`=0.90, `colsample`=0.80, `reg_alpha`=0.10, `min_child_samples`=25	1000 trees, 15 leaves, depth 6, full row/column sampling, `reg_alpha`=0.70, `min_child_samples`=35	+0.0031
CatBoost	depth 6, `l2_leaf_reg`=3, `subsample`=0.80, `rsm`=1.00, `random_strength`=1, `min_data_in_leaf`=1	depth 7, `l2_leaf_reg`=7, `subsample`=0.88, `rsm`=0.85, `random_strength`=0.5, `min_data_in_leaf`=8	+0.0023

TABLE VII
REPRESENTATIVE CANDIDATE LINES AND WHAT THEY TAUGHT US

Candidate line	Local metric	Public score	Role in the project	Main lesson
LightGBM baseline	0.81226	0.80640	Early validation-first baseline	Useful baseline for promotion decisions, but not strong enough as a final competition submission.
Clean CatBoost single model	0.81330	0.81061	Best clean single-model anchor	The most defensible report centerpiece: strong score, simple preprocessing, and high interpretability.
Clean CatBoost reproduction	0.81422	0.80991	Transparent clean reproduction	A respectable public reproduction that still failed to beat the clean anchor.
High-CV CatBoost candidate	0.81905	0.80593	High-CV counterexample	Strongest evidence that local group-aware CV alone is not a dependable public-score ranker near the top of this competition.
Public-best XGB branch	0.80516	0.81716	Best leaderboard branch	Highest public result, but not the strongest scientific baseline; must be explained as a separate competition-side line.

This difference matters because the last XGBoost settings are not presented as the one true peak in local-CV space. We make a narrower claim: the public-best version rests on a visible experiment trail. It started from a compact XGBoost setup shaped by Optuna. After that, preprocessing, selected features, and the `KMeansSMOTE` layout stayed fixed. Near the contest deadline, only tree count, learning rate, and seed were adjusted. The final setting used 950 trees, learning rate 0.0475, and seed 36086, while keeping earlier regularization and sampling rules untouched. Small shifts around those values showed that the branch sits among solid neighbors rather than at an isolated extreme. It is therefore less a lucky guess than a documented trace left by real testing.

What made the last version work became clearer once we rebuilt it piece by piece. Table VIII reports the stage changes. The initial compact preprocessing path reached 0.80445 under 10-fold row-level reconstruction CV. When tested as a filter, the `IsolationForest` diagnostic lowered the score instead of helping. Cutting less useful encoded features lifted the reconstruction score to 0.80755. Under the stricter 5-fold `StratifiedGroupKFold` review, the same branch landed at 0.80516.

Table IX makes the public-side evolution explicit. The tuned XGBoost variant rejected on April 10, 2026 was useful because it taught us that parameter changes alone did not move the needle. The shift came only after we stopped treating XGBoost as a direct solo fix and placed it inside a wider contest-style setup.

We also ran a close local scan around the branch settings to check whether the top public setup held up nearby under ordinary validation. Figure 11 shows the result. Row-level CV slightly preferred fewer effective steps through a lower learning rate and more trees, while the group-aware check leaned toward less dense sampling. Even so, every tested version stayed close in performance, and no option clearly pulled ahead under both views.

This finding shapes the paper's argument. The top public XGBoost version is not a one-of-a-kind local peak. It is better understood as sitting inside a tight cluster of strong choices. Its edge on the leaderboard appears partly because of interactions that cross-validation alone cannot capture. Calling it simply "the best" would stretch beyond what the data show.

Two points follow. First, the last branch did not appear out of nowhere; it marks where earlier rounds of XGBoost testing finally landed. Second, it makes more sense as a competition-side result than as the top scientific standard. Its public success is real, but the paper's most solid argument still rests on the benchmark that put validation first.

H. Where the Two Branches Disagree

The previous subsection explains why the public-best XGBoost version is not a perfect local solution. Another angle is whether it still beats the simpler CatBoost model across the board. Under the same 5-fold `StratifiedGroupKFold` review, it does not. The two branches produce different predictions for 8.49% of training samples, and the streamlined CatBoost line keeps a 0.37-point accuracy lead in that shared evaluation.

Figure 12 shows where the disagreement concentrates. Decks E and G show the biggest split in the cabin area. Around behavior, mismatches pile up in cryosleep-spend gray zones: passengers with missing `CryoSleep` but zero spending, and passengers marked as `CryoSleep=False` while also showing no expenses. These are the cases where preprocessing choices can swing outcomes sharply. XGBoost is not always behind; small pockets such as Deck A and Deck E show a slight edge. The meaning is narrow rather than universal: a handful of choices in the compact XGBoost path sometimes

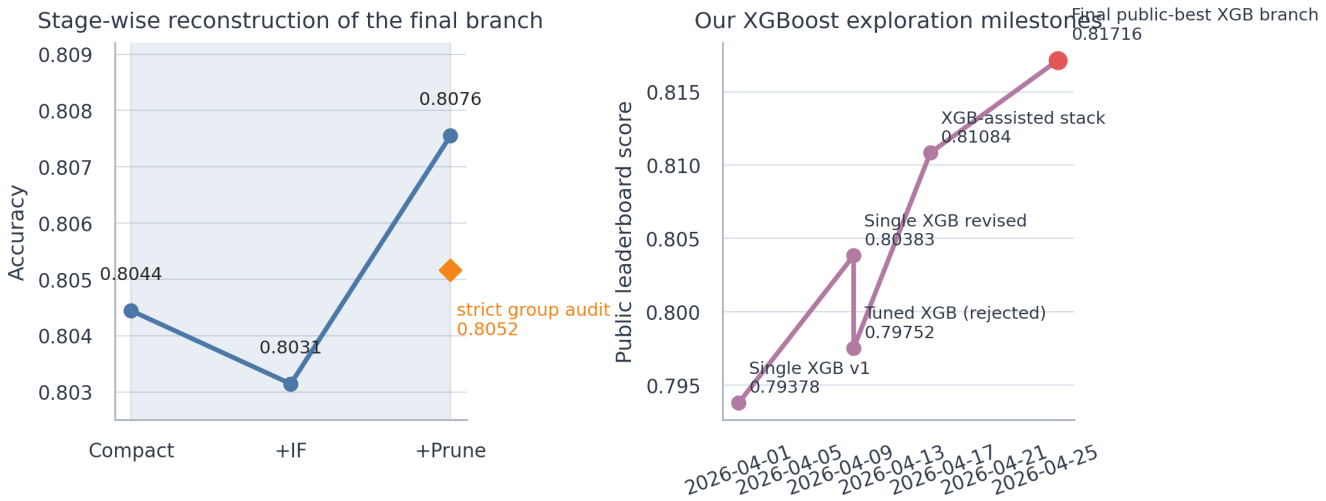

Fig. 10. Left: stage-wise reconstruction of the final XGBoost branch. Right: the public-score trajectory of our XGBoost-centered exploration path from April 1, 2026 to April 26, 2026.

TABLE VIII
STAGE-WISE RECONSTRUCTION OF THE FINAL XGBOOST BRANCH

Stage	Validation view	Accuracy	Std. dev.	Interpretation
Compact preprocessing branch	10-fold row-level reconstruction CV	0.80445	0.01417	A workable but not yet competitive compact XGBoost baseline.
+ IsolationForest diagnostic	10-fold row-level reconstruction CV	0.80314	0.01590	Outlier filtering did not help locally and was not used in the final submitted training matrix.
+ Feature pruning	10-fold row-level reconstruction CV	0.80755	0.01542	Manual pruning recovered a clearer gain and created the final compact branch used for the public-best submission.
Strict group-aware audit	5-fold StratifiedGroupKFold	0.80516	0.00490	Under the stricter report-standard audit, the branch remains weaker than the clean CatBoost anchor despite its public success.

TABLE IX
MILESTONES IN THE XGBOOST EXPLORATION BRANCH

Date	Milestone	Public score	Why it mattered
2026-04-01	Single XGB v1	0.79378	Established an early XGBoost baseline and showed that a straightforward single-model configuration was not yet competitive.
2026-04-10	Single XGB revised	0.80383	Confirmed that XGBoost could improve materially with better preprocessing, even before branch-level redesign.
2026-04-10	Tuned XGB (rejected)	0.79752	Demonstrated that naive tuning did not help and that parameter changes alone were insufficient.
2026-04-16	XGB-assisted stack	0.81084	Showed that XGBoost became useful as a controlled component inside a stronger blended system.
2026-04-26	Final public-best XGB branch	0.81716	Produced the highest public score in the team history and finalized the competition-side narrative.

helps publicly, but it does not outrun the clearer CatBoost path at every turn.

I. Final Kaggle Ranking and Submission History

The public leaderboard snapshot from May 17, 2026 places our team `EAP_Hater@MLW` at Rank 97 out of 2240 teams with a score of 0.81716. That lands the team near the top 4.3% of ranked entries in the downloaded snapshot. Our strongest clean single-model attempt reached 0.81061 and Rank 238.

Figure 13 shows the public-score path over time. Figure 14 places our result within the range of all public leaderboard scores. To meet course requirements, Figure 15 displays selected Kaggle submission milestones pulled through the Kaggle API on May 17, 2026. Clear semantic labels are used in the figure, while the full raw record remains unchanged in the exported files.

VI. DISCUSSION

Three ideas stand out.

First, *Spaceship Titanic* can mislead when the work chases public scores without deeper scrutiny, especially in a scholarly report. Since the data pool is small and widely seen, score shifts may expose quirks of a specific split instead of real model strength. Earlier studies support this concern: too much interaction with public feedback loops creates evaluation risk, not just harmless noise [8].

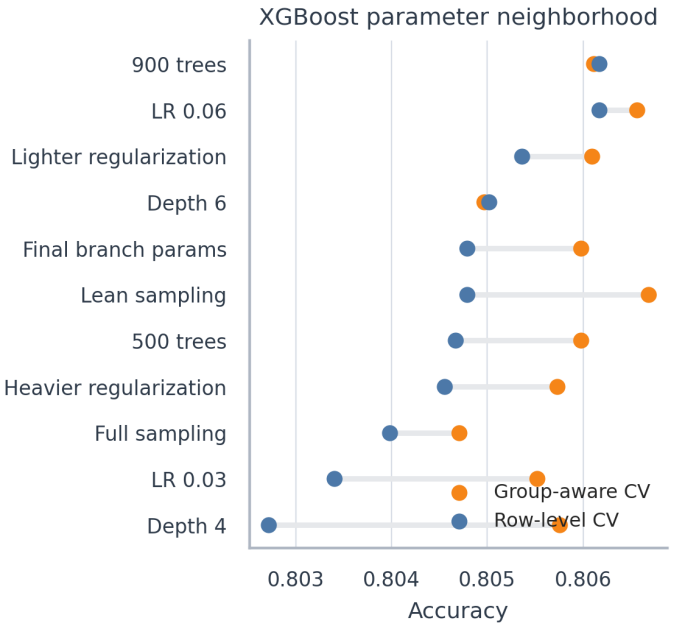

Fig. 11. Focused local neighborhood search around the final XGBoost branch. The public-best configuration sits inside a competitive local plateau rather than as a dominant local optimum.

Second, the best clean solo model is not identical to the model that scored highest on the leaderboard. A clear CatBoost setup works better when explaining results: it shows its logic, stays stable under review, and stands up better to questions. The XGBoost path pushed the team furthest in rankings. Treating them as separate jobs, one for clarity and one for points, removes much of the tension.

Third, the study checks its own results. It uses repeated group-aware testing, separates early clues from later verification, and refuses to chase tiny improvements until broader tests confirm them. For this data, staying strict matters more than small shifts in published numbers.

VII. Threats to Validity

Four issues affect how far the findings can be taken.

First, the public leaderboard is not the full picture. It covers only part of Kaggle's hidden test data, so a visible improvement may not carry over to the private split. This is why the paper does not claim that the top public result is the best model overall.

Second, the competition-side XGBoost branch uses tactics that are more acceptable in contests than in strict research settings, including train-test harmonization and manual feature pruning. We therefore label it clearly as a leaderboard branch rather than the primary scientific conclusion.

Third, the final public XGBoost version includes seed-specific selection during competition work. We therefore list it as the final entry, not as the best setup for every case.

Fourth, the whole analysis happens inside a single contest environment. Although our validation approach is stricter than a basic train-test split, the results still apply specifically to *Spaceship Titanic*. The work should be read as a close look at tabular competition practice, not as a broad claim about all datasets.

VIII. Future Work and Conclusion

If the project continued past the course deadline, chasing a higher public score would not come first. More useful next steps would include tuning while respecting subgroup patterns, examining errors by validation split, checking whether predicted probabilities match real outcomes, and measuring how much feature importance shifts between runs. Regions where CatBoost and XGBoost assign different probabilities also deserve deeper study.

The project met its goals, though not through one path. On the ranking side, the team reached 0.81716 and Rank 97/2240 in the public leaderboard snapshot dated May 17, 2026. On the reporting side, the work built a repeatable validation-first process, tested five model families, documented computational cost, and showed that local checks can differ from live results in either direction. With a well-known dataset such as *Spaceship Titanic*, real insight grows through strict testing and step-by-step ablation. A higher shared ranking matters, but thorough checking matters more.

IX. Code Availability and Contribution Statement

A. Code Availability

The final IEEE paper, generated figures, tables, reproducible notebook, and code are organized in the submitted repository:

https://github.com/Dylan4X/spaceship-titanic-mlworkshop

B. Contribution Statement

Dingrong Xue contributed 34% of the project work, including Kaggle leaderboard experimentation, validation design, model benchmarking, final report writing, experimental refinement, and reproducible code preparation. Fan Xie contributed 33%, including Kaggle leaderboard experimentation, competition-side XGBoost exploration, hyperparameter search, result discussion, and presentation slide preparation. Shijun Cheng contributed 33%, including exploratory data analysis, feature engineering support, interpretation of dataset patterns, visualization preparation, and presentation slide preparation. All members participated in final project review.

Appendix A
Requirement Traceability

Table X maps the course report requirements to concrete evidence in this paper.

Appendix B
Milestone Submission Record

Table XI records the milestone submissions that matter most to the final narrative.

This appendix is included for two practical reasons. First, the course requires evidence of the final Kaggle workflow

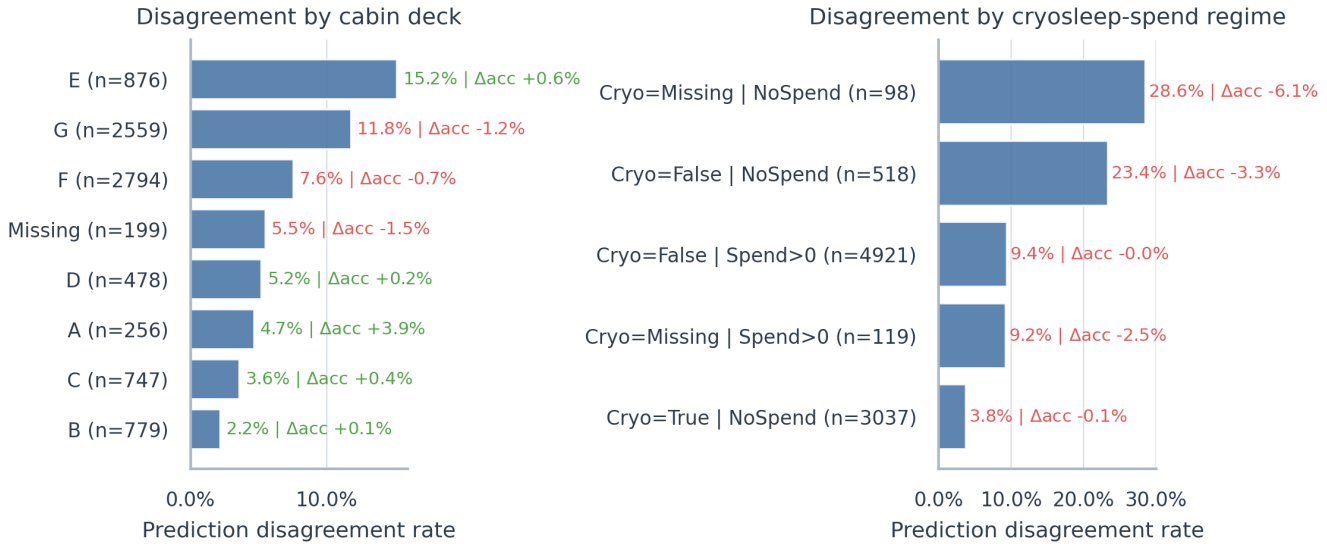

Fig. 12. Shared-audit disagreement between the clean CatBoost line and the compact XGBoost branch. Text annotations report subgroup disagreement rate and local accuracy gap (XGBoost minus CatBoost).

TABLE X
TRACEABILITY FROM COURSE REQUIREMENTS TO EVIDENCE IN THE FINAL PAPER

Course requirement	Evidence in this paper
Abstract	Abstract section on page 1 summarizes the task, methods, and final outcomes.
Introduction	Section I explains the competition setting and the distinction between leaderboard outcome and scientific claim.
Related work / existing techniques	Section II discusses boosting models, validation practice, and representative Kaggle guides.
Methodology	Section IV documents preprocessing, feature engineering, validation design, and the competition-side XGBoost branch.
Experimental study and results analysis	Section V contains the unified benchmark, CV-public alignment analysis, XGBoost branch reconstruction, and final ranking discussion.
Future work and conclusion	Section VIII provides both next steps and final conclusions.
References	The bibliography cites the competition page, model papers, Optuna, and representative Kaggle notebooks.
Contribution statement	Section IX contains a member-level contribution statement with named authors and workload percentages.
Final Kaggle ranking	Section V-E and Figures 13–14 report Rank 97/2240 with public score 0.81716 in the May 17, 2026 snapshot.
GitHub link of the code	Section IX-A provides the repository URL field for the submitted code.
Screenshot(s) of Kaggle submission log(s)	Figure 15 provides a cleaned submission-log snapshot with representative milestones retrieved on May 17, 2026.

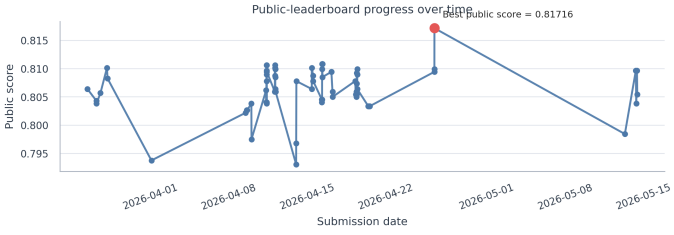

Fig. 13. Public leaderboard progress over time. The best visible submission remains the XGBoost branch submitted on April 26, 2026.

rather than only the final score. Second, the main body of the paper repeatedly distinguishes between clean scientific anchors and competition-side artifacts; the milestone list makes that distinction concrete in submission-history terms.

Read together with Figure 13, this appendix clarifies the project chronology. The April 26, 2026 XGBoost branch remained the best visible public result through the final snapshot on May 17, 2026, while the cleaner CatBoost line remained the report's main scientific anchor. That dual interpretation is

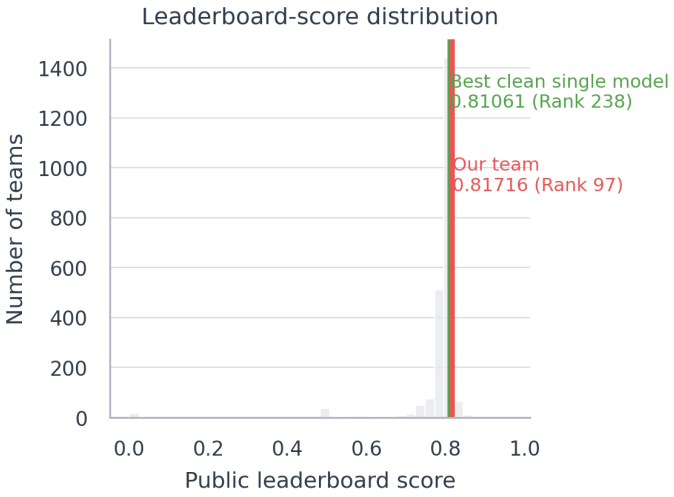

Fig. 14. Leaderboard-score distribution in the public snapshot downloaded on May 17, 2026. The team result and the best clean single-model result are highlighted.

Representative Kaggle submission milestones retrieved on 2026-05-17

Milestone	Submission time	Public score	Role
Clean CatBoost exact line	2026-04-11 09:36	0.81061	Clean anchor
Flaykaer clean reproduction	2026-04-12 16:09	0.80991	Clean reproduction
High-CV CatBoost counterexample	2026-04-12 17:34	0.80593	Validation counterexample
XGB-assisted stack	2026-04-16 11:57	0.81084	Competition-side hybrid
Final public-best XGB branch	2026-04-26 08:10	0.81716	Public-best branch
Late blend attempt	2026-05-14 09:37	0.80967	Late check

Fig. 15. Representative Kaggle submission milestones retrieved on May 17, 2026. Semantic labels are used in place of raw file names; the best visible score is the final public-best XGBoost branch at 0.81716.

TABLE XI
MILESTONE SUBMISSION RECORD USED IN THE FINAL ANALYSIS

Date	Milestone label	Public score	Pipeline type	Why it matters
2026-04-11	Clean CatBoost single model	0.81061	Clean single model	Best clean single-model anchor used for most of the report narrative.
2026-04-12	Clean CatBoost reproduction	0.80991	Clean reproduction	Transparent group-aware CatBoost reproduction that remained interpretable but did not beat the clean anchor.
2026-04-12	High-CV CatBoost candidate	0.80593	High-CV clean candidate	Strong local-CV counterexample showing that public and local metrics can diverge sharply.
2026-04-16	XGB-assisted stack	0.81084	Competition-side hybrid	Demonstrates that XGBoost became useful before the final public-best branch.
2026-04-26	Final public-best XGB branch	0.81716	Competition-side compact branch	Highest public score in the team history and the endpoint of the XGBoost exploration path.
2026-05-14	Late blend attempt	0.80967	Late ensemble check	Confirms that later submissions did not surpass the April 26, 2026 public best.

central to the paper and is therefore documented explicitly here.

REFERENCES

[1] Kaggle, "Spaceship Titanic," online competition page. Available: https://www.kaggle.com/competitions/spaceship-titanic/overview

[2] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[3] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems*, 2017.

[4] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "CatBoost: unbiased boosting with categorical features," in *Advances in Neural Information Processing Systems*, 2018.

[5] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[6] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[7] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proc. 25th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2019, pp. 2623–2631.

[8] A. Blum and M. Hardt, "The Ladder: A reliable leaderboard for machine learning competitions," in *Proc. 32nd Int. Conf. Machine Learning*, 2015, pp. 1006–1014.

[9] S. Cortinhas, "Spaceship Titanic: A complete guide," Kaggle notebook, 2022. Available: https://www.kaggle.com/code/samuelcortinhas/spaceship-titanic-a-complete-guide

[10] Arunklenin, "Space Titanic — EDA — Advanced Feature Engineering," Kaggle notebook, 2024. Available: https://www.kaggle.com/code/arunklenin/space-titanic-eda-advanced-feature-engineering